

HW1: Mid-term assignment report

Alexandre Miguel Rodrigues Cotorobai [107849], v2024-04-09

1 Introduction	1
1.1 Overview of the work	1
1.2 Current limitations	1
2 Product specification	2
2.1 Functional scope and supported interactions	2
2.2 System architecture	2
2.3 API for developers	2
3 Quality assurance	2
3.1 Overall strategy for testing	2
3.2 Unit and integration testing	2
3.3 Functional testing	3
3.4 Code quality analysis	3
3.5 Continuous integration pipeline [optional]	3
4 References & resources	3

1 Introduction

1.1 Overview of the work

This report presents the midterm individual project required for TQS, covering both the software product features and the adopted quality assurance strategy.

BusTicket is a full-stack web application designed for searching bus connections between cities and booking reservations for passengers. The purpose of this application is to provide users with a convenient platform to buy and manage their bus travels efficiently. BusTicket focuses on optimizing usability and functionality to guarantee an efficient experience for travelers.

1.2 Current limitations

- The implementation of searching by email is almost complete. I have finished creating the endpoint and the backend, but I didn't have time to create the frontend.
- The project is not deployed in the cloud.

- About the optional of implementing the logic of the booking service for a more realistic business case I only implemented the referred search by email (partially).

2 Product specification

2.1 Functional scope and supported interactions

This app is intended for anyone who has the need to buy bus tickets online
Two base use cases:

Case 1: Making a reservation for a trip: The user accesses the platform, selects origin, destination, and date, and is shown a list of buses that meet these conditions. After choosing, they enter their details and purchase the ticket.

Case 2: Consulting a previously purchased ticket: Having already purchased the ticket and knowing its code (token), it is possible to search and consult the info of that ticket.

2.2 System architecture

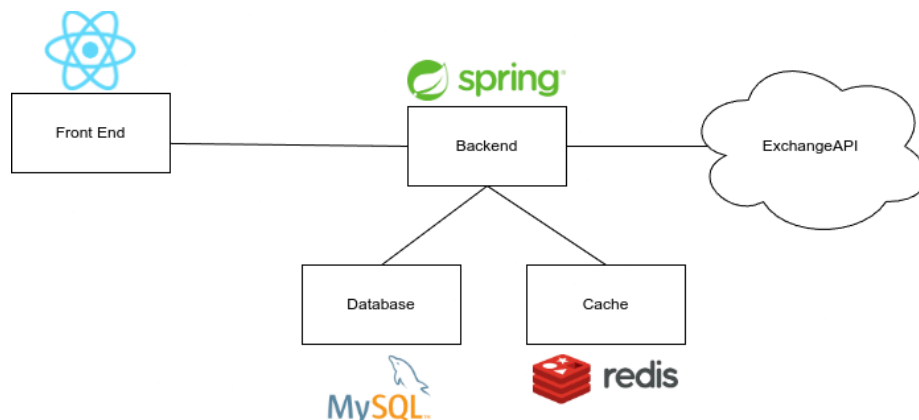


Fig 1 - Architecture

The backend was built using Spring Boot, that is where all the logic was implemented. For the ExternalAPI I used “The Twelve Data API”, a free (limited) forex API, which was integrated to fetch currencies and exchange rates. MySQL for the database and redis as a cache to avoid overloading the API.

2.3 API for developers

ticket-rest-controller		^
POST	/api/tickets	▼
GET	/api/tickets/{id}	▼
GET	/api/reservations	▼
trip-rest-controller		^
GET	/api/trips	▼
GET	/api/trips/{id}	▼
currency-rest-controller		^
GET	/api/stats	▼
GET	/api/currency/{currency}	▼
GET	/api/currencies	▼

Fig 2 - Endpoints

GET /api/tickets/{id}

Input: Path parameter id representing the unique identifier of the ticket.

Output: Returns a ResponseEntity containing either:

HttpStatus.NOT_FOUND if no ticket is found with the specified id.

HttpStatus.OK along with the Ticket object if the ticket is found.

POST /api/tickets

Input: A JSON representation of the Ticket object in the request body.

Output: Returns a ResponseEntity containing either:

HttpStatus.CREATED along with the created Ticket object if success.

HttpStatus.BAD_REQUEST if an exception occurs during the purchase process.

GET /api/reservations

Input: Optional query parameter email representing the email address of the user whose reservations are to be retrieved.

Output: Returns a ResponseEntity containing a list of Ticket objects associated with the specified email address (if provided) or all reservations if no email is provided. The response status is HttpStatus.OK.

GET /api/trips

Inputs: This endpoint accepts three optional query parameters:

startLocation: Represents the starting location of the trip.

endLocation: Represents the ending location of the trip.

dateTime: Represents the date and time of the trip.

Output: Returns a ResponseEntity containing a list of Trip objects based on the provided criteria. If no trips are found matching the criteria, it returns an

HttpStatus.NOT_FOUND status. If trips are found, it returns an HttpStatus.OK status along with the list of Trip objects.

GET /api/trips/{id}

Inputs: Path parameter id representing the unique identifier of the trip.

Output: Returns a ResponseEntity containing either:

HttpStatus.NOT_FOUND if no trip is found with the specified id.

HttpStatus.OK along with the Trip object if the trip is found.

GET /api/currencies

Inputs: This endpoint does not expect any input parameters.

Outputs: Returns a ResponseEntity containing a list of strings representing all available currencies. If no currencies are found, it returns an HttpStatus.NOT_FOUND status. If currencies are found, it returns an HttpStatus.OK status along with the list of currency strings.

GET /api/currency/{currency}

Inputs: Path parameter currency representing the code of the currency to retrieve.

Outputs: Returns a ResponseEntity containing either:

HttpStatus.NOT_FOUND if the specified currency is not found.

HttpStatus.OK along with the Currency object representing the exchange rate of the specified currency if found.

GET /api/stats

Inputs: This endpoint does not expect any input parameters.

Outputs: Returns a ResponseEntity containing a Map<String, Integer> representing statistics related to the API. The map keys represent different statistics API_CALLS, CACHE_HITS, API_MISSES.

3 Quality assurance

3.1 Overall strategy for testing

I mainly opted for Test-Driven Development (TDD) throughout the development process. It greatly helped in planning how I wanted the application to function, as I defined the expected behavior in tests and then implemented based on those tests. This approach was especially useful when dealing with external APIs.

Used JUnit for writing tests, and Mockito/MockMvc were used to isolate what was being tested. Also, I utilized AssertJ and RestTemplates for testing purposes and for frontend, I used Cucumber and Selenium.

3.2 Unit and integration testing

I used UnitTests to test methods of the entities used, as well as the services (Currency, Ticket, Trip) in which I relied on Repository mocks to prevent them from depending on the repository behavior.

Repositories were tested in isolation to ensure there was a connection and data persistence.

Integration Tests were implemented on all endpoints to ensure they were functioning as expected.

Whenever it was necessary to verify the behavior of a component in isolation, mocks were used so that it did not depend on external factors.

3.3 Functional testing

Using Selenium, I conducted user-facing tests that simulated user interactions, serving as acceptance tests to validate the product concept. This was where its practical usage was proven.

I used Selenium with Cucumber.

```
public class TicketPurchaseSteps {
    private WebDriver driver;
    private HomePage homePage;
    private BuyPage buyPage;
    private DetailsPage detailsPage;

    private String ticketcode;

    @Given("I am on homepage")
    public void the_user_is_on_the_homepage() {
        driver = WebDriverManager.chromedriver().create();
        homePage = new HomePage(driver);
    }

    @When("I filter the origin to {string}")
    public void i_filter_the_origin(String origin) {
        homePage.setOrigin(origin);
    }

    @When("I filter the destination to {string}")
    public void i_filter_the_destination(String destination) {
        homePage.setDestination(destination);
    }
}
```

Fig 3 - TicketPurchaseSteps.java

3.4 Code quality analysis

The tool used for static code analysis was Sonarqube. This tool identified potential issues, vulnerabilities, and coverage. It helped me to have my code clean.

The final version of the code passed the quality gate, with an coverage percentage of 95.7% with no issues identified.

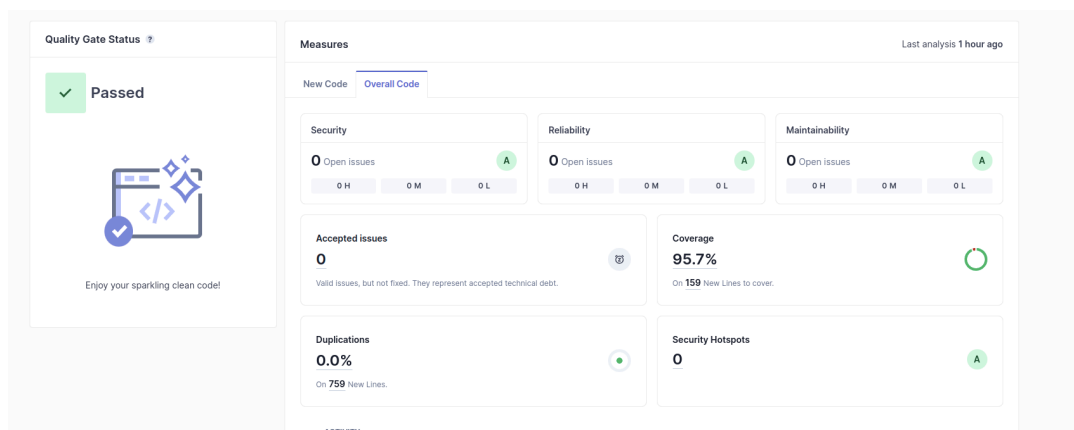


Fig 4 -Dashboard

It was used since the beginning of the project, accompanying its development. This are some of the most recent analysis:

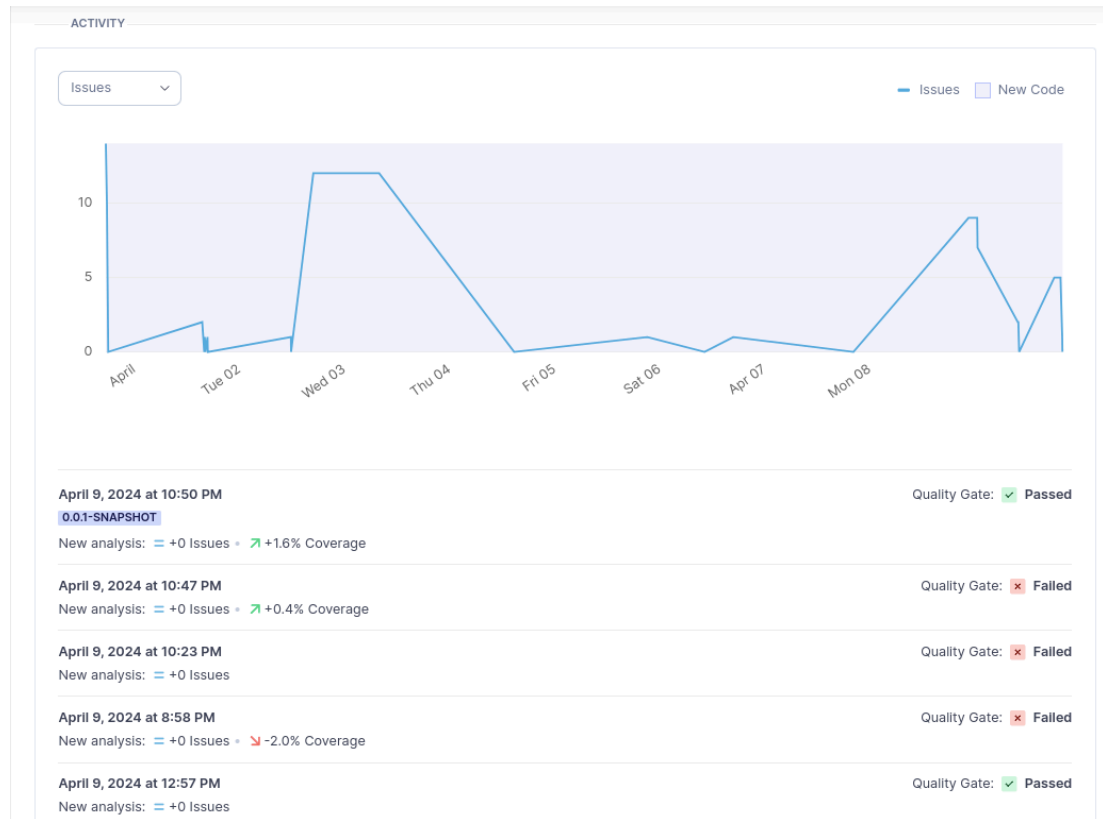


Fig 5 - History

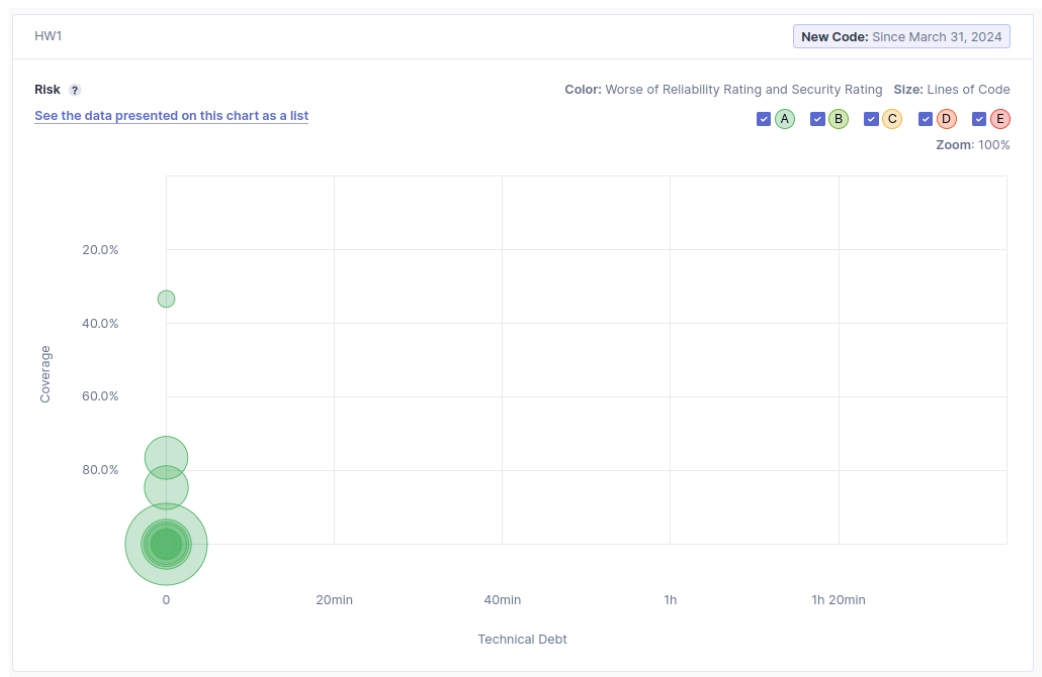


Fig 6 - Technical Debt

3.5 Continuous integration pipeline [optional]

The CI pipeline used was GitHub Actions, executing all created tests except for Selenium tests

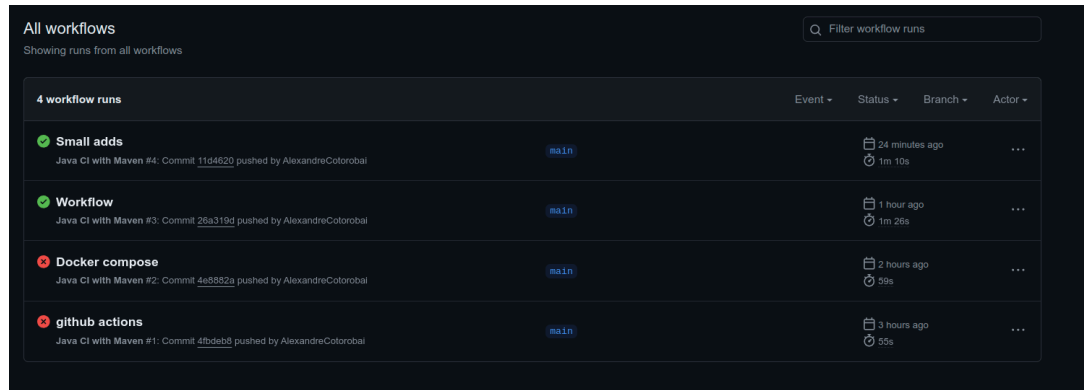


Fig 7 - Github Actions Workflows

```
name: Java CI with Maven

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    name: Build
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: HW1/bustickets

    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven
      - name: Docker dbs
        run: docker run --name mysql5tqs -e MYSQL_ROOT_PASSWORD=password -e
      - name: Docker Redis
        run: docker run --name cache -p 6378:6379 -d redis
      - name: Build with Maven
        run: mvn -B package --file pom.xml -Pno-selenium
```

Fig 8 - Workflow file

4 References & resources

Project resources

Resource:	URL/location:
Git repository	https://github.com/AlexandreCotorobai/TQS_107849

Video demo	https://drive.google.com/drive/folders/1XJVH2jEjKDq9Y45YMGTro4_dKjoSaD3?usp=sharing
CI/CD pipeline	https://github.com/AlexandreCotorobai/TQS_107849/actions

Reference materials

<https://rapidapi.com/twelve-data1-twelve-data-default/api/twelve-data1>

<https://www.baeldung.com/spring-boot-testing>

<https://www.baeldung.com/thymeleaf-in-spring-mvc>

<https://www.geeksforgeeks.org/redis-cache-in-java-using-jedis/>

<https://www.baeldung.com/java-spring-mockito-mock-mockbean>

<https://www.baeldung.com/injecting-mocks-in-spring>

<https://mvnrepository.com/>